

ProofForge V2

One source, multi-chain — no silent drift

Hackathon Research Final · 7 min

DaviRain / ProofForge

Multi-chain today is expensive and risky

- One dApp → Solidity, Rust/Solana, Rust/NEAR, Noir, Move...
- Each target re-implements business logic → **semantic drift**
- Re-audit, re-test, re-document for every chain
- Toolchain / version mismatch → "works on my machine"

A growing, real demand

- ZK rollups, L2s, appchains, cross-chain apps exploding
- Teams want to deploy on the chain that users prefer
- No widely-adopted "write once, run anywhere" for smart contracts
- Security audit market is large and **per-chain**

Our product: a portable compiler

- One `program ... where` source
- `--target evm | solana | near | noir`
- Compiler guarantees semantics **or fails closed**
- No top-level contract / circuit / zkVM label

```
program Counter where
  state count : UInt64

  init(initial : UInt64) do
    count := initial

  entry increment(delta : UInt64) : UInt64 do
    count := count + delta
    return count

  view get() : UInt64 do
    return count
```

User flow: write, check, build, inspect

```
proof-forge-next check Counter.lean
proof-forge-next build Counter.lean --target evm
proof-forge-next inspect --target evm
# OutputSet: ABI, bytecode, manifest
```

- `check` validates without choosing a target
- `build` emits target artifacts + provenance manifest
- `inspect` lists diagnostics and support decisions
- `deploy` / `prove` / `verify` are **explicit** follow-up commands

ProofForge V2 — Architecture Overview

One portable program source → target-neutral semantics → target-owned materialization

Author / CI (trusted input boundary)

program ... where
(Lean portable DSL)

CLI: --target / profiles
proof-forge-next

No top-level kind: not contract | circuit | zkVM in source

proof-forge-next (compiler, not a chain VM)

Source Frontend
Lean Syntax
+ preflight
→ Source.Program

Semantic Engine
Typed → Semantic
+ Requirements

Support Resolver
exact SupportClaim
fail closed

Materializers
Plan → TargetIR
(associated types)

Artifact Pipeline
atomic OutputSet
+ provenance

Phase-1 targets (target-owned Plan; INV: semantics unchanged by --target)

EVM
Yul → solc bytecode
runtime-validated-alpha

Solana
.sbpf-plan + IDL
plan-only (no ELF yet)

NEAR
WAT → Wasm
wasm-validated-alpha

Noir
.nr packages
source-only (no proof)

Outside compiler (untrusted / explicit)

Official packagers
solc / wat2wasm / ...

Local runtime
Anvil / sandbox / ...

deploy / prove / verify
(never implicit)

Network RPC / keys
not compiler inputs

Invariants: INV-001 source layers ignore TargetId · INV-002 unequal map → reject · INV-005 no fallback success · INV-010 clean-room independent of active/ (v1 archive)

Technical differentiation

What others do

What we do

Best-effort transpiler

Exact requirement → SupportClaim resolver

Source coupled to target

Target-neutral `Semantic.Program`

Plan as string / JSON

Typed `Plan` + `TargetIR` per target

Silent fallback

Fail-closed diagnostics

Trust environment

Deterministic, reproducible, clean-room build

What works now?

Target	Evidence	Not yet claimed
EVM	<code>solc</code> bytecode + Anvil runtime	Complete EVM backend
Solana	typed <code>.sbpf-plan</code> + IDL	sBPF object / ELF / runtime
NEAR	raw-u64 WAT/Wasm via <code>wat2wasm</code>	Sandbox receipt
Noir	target-owned Plan + relation IR + <code>.nr</code>	ACIR / proof / VK

Honest maturity: all artifacts share one semantic hash.

AI & Web3: where we fit

- Deeply Web3: smart contracts, ZK circuits, multi-chain accounts
- Not a wrapper: target-owned Plan/IR + exact capability matching
- AI is **not** used in the compiler core today
- Instead, we provide a **deterministic verification layer** — useful whether code is written by humans or AI assistants

Vitalik: the same direction

- A high-level language compiled to Lean
- For human-readable definitions & theorems
- Exactly the formal backbone we are building
- AI-generated proofs need a readable, verifiable layer



vitalik.eth
@VitalikButerin



🔄 翻译自 英语 显示原文 🗨

一种似乎真的值得尝试创建的“高级编程语言”新类型，是一种被编译到 Lean（或 HOL，或.....）的语言，它专门致力于让人类尽可能友好地阅读定义和定理。

不是证明——因为证明的关键只在于证明是正确的——只是定义和定理。

预期的使用场景是，AI 输出一大堆证明，你试图让任何阅读输出的读者尽可能容易地理解已经证明的确切声明是什么。

评价此翻译： 👍 🗨

下午11:01 · 2026年7月21日 · 23.8万 查看

💬 266

↺↻ 143

❤️ 1,288

🔖 202



相关 ∨

查看引用 >

Business model & ecosystem

- **Open-source compiler core** → adoption and community trust
- **Enterprise services:** multi-chain audits, CI gate, SBOM reporting
- **Toolchain integrations:** provers, SDKs, deployment pipelines
- **Ecosystem value:** fewer audits, faster cross-chain launches, lower regression risk

Why us, not a transpiler or bridge?

- **Transpilers:** often best-effort, silent semantic drift
- **Bridges:** runtime layer, not source-level semantics
- **ProofForge:** fail-closed, exact-version, reproducible build
- Target picks the form, but **cannot change the meaning**

ProofForge V2 - Target Landscape

Family is a reading view only; compiler decisions use multi-axis TargetDescriptor + SupportClaim

Phase 1 — implement

evm · contract VM
runtime-validated-alpha

solana · SVM accounts
plan-only

near · Wasm host
wasm-validated-alpha

noir · circuit
source-only

Design / research only — no product backend claim

cosmwasm · soroban · icp
Wasm hosts (own Plans)

openvm · zkVM

aleo · psy
ZK app chains

shared Wasm = AST/encode
never shared host Plan

Maturity ladder (no skipping)

research

design

implemented

runtime-
validated

network / proof-
validated

Team & next steps

- D0 closeout: independent compiler, docs, specs, SBOM
- D1: full parser + type/effect system
- D2: structs, events, fn calls, proof references
- Phase 1: EVM runtime, Solana ELF, NEAR sandbox, Noir proof
- Design-only: CosmWasm, Soroban, ICP, OpenVM, Aleo, Psy

Let's build the common source of truth

ProofForge V2

- Repo: `github.com/DaviRain-Su/proof_forge`
- Docs: `docs/01-prd.md` · `docs/02-architecture.md`
- Try: `just build && just test`

Q & A